Relatório 1 de Acompanhamento

Identificação de Sistemas e Controle Preditivo Baseado em Modelos de Espaço de Estados Estruturados (SSMs)

Gustavo Jae Min Yu – Nº USP 13782762
Ian Seo Takose – Nº USP 13680592
Vinicius Kenji Yamada – Nº USP 13782863

14 de setembro de 2026

Trabalho de Conclusão de Curso – 2º semestre de 2026
Orientação: Felipe Miguel Pait.
Primeira apresentação: 15 de setembro de 2026.

1 Resumo da proposta e do estágio anterior

A proposta do projeto é investigar o uso de modelos de espaço de estados estruturados (*Structured State Space Models*, SSMs) na identificação de sistemas e, posteriormente, sua possível aplicação ao Controle Preditivo Baseado em Modelo (MPC). A identificação consiste em obter um modelo do comportamento de um sistema a partir de dados de entrada e saída. No MPC, um modelo é utilizado para prever esse comportamento e auxiliar na escolha das ações de controle.

O interesse pelo tema surgiu a partir das arquiteturas S4 e Mamba, utilizadas em aprendizado de máquina para processar sequências [1, 2]. Pretende-se compreender seus princípios e investigar sua adequação à modelagem de sistemas dinâmicos. Até esta primeira entrega, o projeto permanece em fase de pesquisa introdutória, sem resultados experimentais a apresentar.

2 Atividades realizadas e resultados obtidos nesta fase

As atividades realizadas até o momento se limitaram a uma **pesquisa básica sobre o funcionamento do S4 e do Mamba**. O objetivo inicial foi obter uma visão geral dessas arquiteturas e de como elas processam informações ao longo de uma sequência. O estudo ainda não abrange o domínio de sua formulação matemática ou de sua implementação.

Em nível introdutório, o S4 utiliza um modelo de espaço de estados com estrutura específica para representar dependências ao longo da sequência [1]. O Mamba introduz um mecanismo seletivo, dependente da entrada, que permite ajustar a forma como informações são preservadas ou descartadas [2]. Essas ideias constituem o ponto de partida do estudo, a ser aprofundado nas próximas etapas.

O resultado desta fase é uma familiarização inicial com as duas arquiteturas e a identificação dos conhecimentos necessários para avançar. **Os métodos clássicos de identificação e a formulação do MPC ainda serão estudados.** Não foram desenvolvidos modelos computacionais, treinamentos, comparações de desempenho ou simulações de controle.

2.1 Noções iniciais: as três visões do S4

O material introdutório de Bourdois [3] apresenta três formas de compreender o componente de espaço de estados do S4: contínua, recorrente e convolucional. Elas ajudam a relacionar a descrição matemática do modelo com seu uso computacional. A síntese abaixo se restringe ao componente linear e invariante no tempo, com intervalo de discretização fixo [1, 4].

Nas expressões, u representa a entrada, x o estado interno que armazena informações do histórico e y a saída. Para simplificar, omite-se o termo de passagem direta da entrada à saída (Du).

Visão contínua: evolução do estado ao longo do tempo

O ponto de partida é uma equação diferencial que descreve como o estado varia. A matriz A determina a dinâmica do estado, B descreve a influência da entrada e C transforma o estado em saída [3, 4].

$$x'(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t).$$

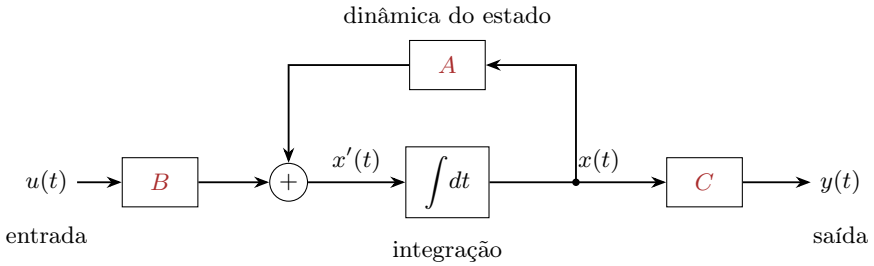


Figura 1: Visão contínua: a soma $Ax(t) + Bu(t)$ determina a derivada do estado, que é integrado para produzir $x(t)$. Omitiu-se o termo $Du(t)$, conforme a simplificação adotada no texto. Esquema baseado em [1, 4].

Essa visão aproxima o modelo da representação de sistemas dinâmicos usada em controle. Para processar amostras no computador, passa-se a uma representação em passos discretos por meio da discretização [4].

Visão recorrente: processamento de uma amostra por vez

Após a discretização, o estado em cada passo é calculado a partir do estado anterior e da nova entrada. As matrizes A_d e B_d representam os parâmetros discretizados [4].

$$x_k = A_d x_{k-1} + B_d u_k, \quad y_k = Cx_k.$$

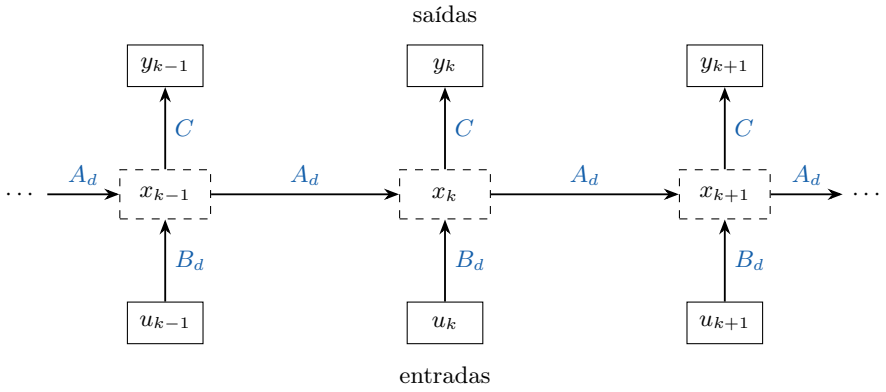


Figura 2: Visão recorrente: em cada passo, o estado reúne a contribuição do estado anterior, via A_d, e da entrada atual, via B_d. A matriz C produz a saída. Esquema baseado em [1, 4].

O estado funciona como uma memória atualizada a cada amostra. Essa forma é conveniente para processar dados que chegam sequencialmente, sem recalcular todo o histórico a cada nova entrada [3].

Visão convolucional: cálculo sobre uma sequência

Com estado inicial nulo antes da primeira entrada, a recorrência linear pode ser reescrita como uma convolução. Um núcleo K reúne os pesos que relacionam as entradas atuais e passadas à saída [1, 4].

$$K = \left(CB_d,\ CA_dB_d,\ CA_d^2B_d,\ \ldots\right), \qquad y = K * u.$$

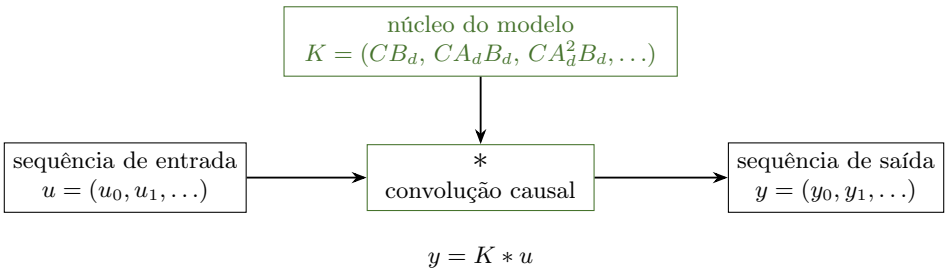


Figura 3: Visão convolucional: o núcleo K combina as contribuições das entradas para calcular a sequência de saídas. Essa representação permite cálculo em paralelo e corresponde à recorrência linear com estado inicial nulo e parâmetros fixos. Esquema baseado em [1, 4].

O símbolo $*$ indica convolução. Essa forma permite calcular as saídas em paralelo quando a sequência está disponível, favorecendo o treinamento. Sob essas condições, recorrência e convolução calculam o mesmo resultado por procedimentos diferentes [3, 4].

Distinção em relação ao Mamba: seu mecanismo seletivo torna parâmetros dependentes da entrada. Por isso, essa equivalência com um único núcleo de convolução fixo não se estende diretamente ao componente seletivo do Mamba [2].

2.2 HiPPO: uma base para a memória do S4

O HiPPO (*High-Order Polynomial Projection Operator*) trata de como representar o histórico de um sinal usando um número limitado de valores. Em vez de armazenar todas as amostras, mantém coeficientes de uma aproximação por polinômios, atualizados conforme novos dados chegam [5].

De forma ilustrativa, o histórico da entrada até o instante t pode ser aproximado por

$$\widehat{u}_t(\tau) = \sum_{n=0}^{N-1} c_n(t)\, p_n^{(t)}(\tau), \qquad 0 \le \tau \le t.$$

Aqui, N é a quantidade de coeficientes armazenados, $p_n^{(t)}$ são os polinômios da base e $c_n(t)$ são os coeficientes que resumem o passado. A importância atribuída a cada trecho do histórico depende da ponderação escolhida; a aproximação não preserva todos os detalhes do sinal [5].

O HiPPO fornece regras de atualização desses coeficientes na forma de um sistema dinâmico. No S4, essa construção fundamenta a escolha e a inicialização de matrizes que favorecem a representação de dependências temporais longas. O S4 também explora a estrutura dessas matrizes para tornar o cálculo eficiente [1, 5].

Assim, o HiPPO complementa as três visões: elas descrevem formas de representar e calcular o modelo, enquanto o HiPPO orienta a construção de sua memória. Sua apresentação aqui é conceitual; a derivação das matrizes e sua implementação permanecem como aprofundamentos futuros.

3 Análise de complexidade computacional e de memória

Esta seção acrescenta uma análise teórica preliminar para justificar o interesse pelas SSMs. As ordens de complexidade abaixo são obtidas pela contagem das operações e dos valores armazenados nas representações discutidas; não constituem resultados experimentais do projeto.

3.1 Notação e hipóteses da comparação

Sejam L o comprimento da sequência, d a quantidade de canais internos e N a dimensão do estado por canal, coerente com a notação usada na apresentação do HiPPO. Consideram-se uma camada, uma sequência por vez e precisão numérica fixa. Fatores constantes de expansão interna são omitidos. A notação $O(\cdot)$ expressa um limite superior assintótico; dizer que um custo é constante em L não significa que ele seja pequeno ou independente de d e N.

A memória é separada em: parâmetros do modelo, ativações usadas no treinamento e memória persistente do histórico durante a inferência. As comparações de memória do histórico excluem parâmetros, entradas e saídas já consumidas. Se todas as saídas forem guardadas, seu armazenamento acrescentará $O(Ld)$ mesmo para uma SSM recorrente. Para um lote de b sequências, os custos associados aos dados e estados são multiplicados por b; os parâmetros são compartilhados.

3.2 Por que a estrutura do estado importa

Considere inicialmente um canal, com entrada e saída escalares e estado de dimensão N. Na atualização $x_k = A_d x_{k-1} + B_d u_k$, uma matriz A_d densa requer $O(N^2)$ operações por passo; a entrada e a leitura Cx_k requerem $O(N)$. Logo, processar L amostras custa $O(LN^2)$, e armazenar a transição densa exige $O(N^2)$ parâmetros. Embora apenas o estado atual, com $O(N)$ valores, seja necessário na inferência, a atualização pode ser cara.

Se a transição for diagonal ou admitir aplicação em tempo linear por uma representação estruturada de posto fixo, a multiplicação passa a custar $O(N)$. Para d canais independentes no núcleo temporal, isso resulta em

$$T_{\text{passo}} = O(dN), \qquad T_{\text{sequência}} = O(LdN), \qquad M_{\text{estado}} = O(dN).$$

Esses custos pressupõem que a estrutura seja efetivamente explorada, incluindo na representação discretizada, e que a preparação dos parâmetros fixos seja feita previamente. Não se aplicam automaticamente a qualquer modelo escrito em espaço de estados. O S4 constrói uma parametrização estruturada para viabilizar esses cálculos [1].

Uma camada completa também pode misturar canais por projeções densas de dimensão proporcional a d. Cada projeção custa $O(d^2)$ por amostra e armazena $O(d^2)$ parâmetros. Assim, uma camada recorrente com essas projeções tem custo $O(L(dN + d^2))$, mantendo crescimento linear em L para dimensões fixas.

3.3 Sequência completa: S4, Mamba e atenção

No S4 linear e invariante no tempo, aplicar um núcleo de comprimento L por convolução direta custa $O(L^2)$ por canal. Usando FFT e preenchimento com zeros para obter a convolução linear causal, a aplicação custa $O(L \log L)$ por canal. A construção do núcleo também tem custo: o algoritmo estruturado do S4 apresenta custo quase linear em $N + L$, com fatores logarítmicos [1]. Portanto, $O(dL \log L)$ descreve a *aplicação de um núcleo já disponível*, e não toda a construção e execução da camada. Denotando o custo de construção por T_K, obtém-se

$$T_{\text{S4,conv}} = T_K + O(dL \log L + Ld^2).$$

Entradas, saídas e núcleos da convolução ocupam $O(Ld)$ valores, além dos parâmetros e do espaço de trabalho da construção do núcleo.

O Mamba utiliza transições seletivas e um *scan* paralelo, pois os parâmetros dependentes da entrada impedem o uso de um único núcleo fixo. Seu núcleo seletivo realiza $O(LdN)$ operações; projeções acrescentam termos como $O(Ld^2)$. A implementação combina fusão de operações e recomputação para evitar guardar todos os estados expandidos na memória principal da GPU [2]. Guardá-los explicitamente exigiria $O(LdN)$ valores. Isso não torna constante a memória de treinamento: ativações associadas às posições da sequência continuam crescendo com L.

Como referência, na atenção densa, Q, K, V possuem L linhas e largura total d. Os produtos $QK^\top$ e a combinação com V custam $O(L^2 d)$; as projeções custam $O(Ld^2)$. A implementação convencional materializa escores de tamanho $L \times L$ por cabeça. Com número fixo de cabeças, isso exige $O(L^2 + Ld)$ memória de ativações. FlashAttention evita armazenar essa matriz completa e permite memória linear em L, mas mantém a ordem quadrática do trabalho aritmético da atenção densa [6].

No treinamento, o cálculo dos gradientes acrescenta trabalho da mesma ordem assintótica dos operadores considerados, nas implementações usuais. O pico de memória depende das ativações salvas, da recomputação e do otimizador. Se P é o número de parâmetros, pesos, gradientes e estados de otimizadores usuais acrescentam $O(P)$ memória, independente de L. Por isso, uma comparação justa deve especificar implementação e regime de execução.

3.4 Inferência incremental e memória do histórico

A tabela compara a chegada de uma nova amostra após um histórico de comprimento L. Para a atenção, considera-se atenção causal densa com cache convencional de chaves e valores (*KV cache*), sem janela limitada ou compressão. Para as SSMs, considera-se a forma recorrente estruturada, com preparação dos parâmetros fixos já realizada.

Modelo / execução	**Computação por nova amostra**	**Memória do histórico**	**Computação para L amostras**
Atenção densa com cache	$O(Ld + d^2)$	$O(Ld)$	$O(L^2d + Ld^2)$
S4 recorrente estruturado	$O(dN + d^2)$	$O(dN)$	$O(LdN + Ld^2)$
Mamba recorrente	$O(dN + d^2)$	$O(dN + dw)$	$O(LdN + Ld^2)$

No Mamba, w é a largura fixa da convolução local; seu buffer ocupa $O(dw)$ e seu custo $O(dw)$ por passo está absorvido na tabela para w fixo [2]. Somar o trabalho da atenção ao longo da sequência produz o termo quadrático, pois $1 + 2 + \cdots + L = L(L + 1)/2$.

Mantendo d, N e w fixos, S4 e Mamba têm custo $O(1)$ por nova amostra e memória persistente $O(1)$ em relação ao histórico; processar as L amostras ainda exige $O(L)$ trabalho. Na atenção considerada, tanto o custo por nova amostra quanto o cache crescem com L. Por exemplo, duplicar um histórico longo aproximadamente duplica o cache e o termo de atenção por passo, enquanto o tamanho do estado da SSM permanece igual. Essa comparação assintótica não determina o ponto em que a SSM passa a ser mais rápida ou consumir menos memória em uma implementação concreta.

3.5 Vantagens para o projeto e limites da análise

As contagens anteriores motivam três benefícios potenciais para identificação de sistemas e controle:

- **Processamento contínuo com memória limitada:** o modelo atualiza um estado de dimensão fixa conforme chegam medições, favorecendo aplicações com restrições de memória.
- **Custo por amostra independente do histórico:** a inferência recorrente facilita estimar o orçamento computacional do modelo ao longo de uma operação prolongada.
- **Uso de sequências longas:** o treinamento pode explorar convoluções no S4 e *scan* no Mamba, sem a comparação explícita entre todos os pares de posições da atenção densa.

Essas vantagens não são exclusivas frente a todos os métodos: modelos clássicos em espaço de estados e RNNs também podem manter memória fixa na inferência. Além disso, comprimir o passado em um estado finito pode perder informações relevantes. A qualidade das previsões, a estabilidade e a adequação aos sinais do projeto precisam ser avaliadas; a complexidade, isoladamente, não demonstra superioridade de identificação.

Para um horizonte de previsão H, uma simulação recorrente do modelo custa $O(H(dN + d^2))$ sob as mesmas hipóteses. Esse é o custo de *uma trajetória prevista*, não o custo total do MPC, que inclui otimização, restrições, eventuais gradientes e múltiplas avaliações do modelo. Logo, a economia no modelo preditivo é uma motivação a investigar, sem garantir execução do controlador em tempo real.

Como avaliação futura, propõe-se variar L e registrar erro de previsão, latência por amostra e pico de memória, separando treinamento e inferência. Hardware, precisão numérica, tamanho do lote e dimensões dos modelos deverão ser documentados para confrontar as tendências teóricas com as medições.

4 Próximos passos e delimitação do projeto

A próxima etapa será aprofundar o estudo de S4 e Mamba e iniciar o aprendizado dos fundamentos de identificação de sistemas e controle preditivo. Com apoio do orientador, serão escolhidos um sistema simples para estudo, uma arquitetura inicial, as ferramentas computacionais e um método de comparação. Essas escolhas ainda não estão concluídas.

5 Cronograma detalhado de resultados esperados

O planejamento abaixo considera o estágio introdutório atual e o período de setembro a dezembro de 2026. As atividades futuras estão organizadas por entregas, sujeitas ao calendário da disciplina e aos ajustes com o orientador.

Período	Atividades previstas e resultados esperados
14 a 20/09	Encaminhar a definição da orientação e discutir a viabilidade da proposta. **Resultado esperado:** prioridades de estudo e escopo preliminar para o semestre.
21/09 a 04/10	Aprofundar o estudo introdutório de S4 e Mamba; iniciar o estudo de identificação de sistemas e MPC. **Resultado esperado:** resumo dos conceitos básicos, das três visões do S4, do papel do HiPPO e das dúvidas a esclarecer.
05 a 18/10	Escolher um sistema simples, as ferramentas e um método de comparação; estudar seu funcionamento. **Resultado esperado:** plano do primeiro experimento, com entradas, saídas e critérios de avaliação definidos.
19/10 a 01/11	Preparar os dados simulados e executar um exemplo inicial da arquitetura escolhida. **Resultado esperado:** ambiente de trabalho organizado e primeiras previsões, com registro das dificuldades.
02 a 15/11	Ajustar o modelo e implementar a comparação escolhida. **Resultado esperado:** gráficos de previsões e dados de teste, medidas de erro e análise preliminar.
16 a 29/11	Aprofundar o estudo de MPC e avaliar a integração do modelo. **Resultado esperado:** definição da estratégia de integração e, se viável, uma primeira simulação de controle.
30/11 até a entrega final	Organizar os resultados e as limitações; revisar o relatório e preparar a apresentação. **Resultado esperado:** documentação do trabalho realizado e dos próximos passos, no prazo oficial da disciplina.

A prioridade do semestre será construir a base teórica e realizar uma avaliação inicial de modelagem. O uso do modelo no MPC permanece como objetivo posterior do projeto. Caso essa integração demande mais tempo, o relatório registrará o estágio alcançado e as etapas pendentes, conforme alinhamento com o orientador. A divisão das atividades entre os integrantes será definida após a delimitação do escopo.

Referências de apoio

[1] GU, A.; GOEL, K.; RÉ, C. *Efficiently Modeling Long Sequences with Structured State Spaces.* ICLR, 2022. Disponível em: https://arxiv.org/abs/2111.00396.

[2] GU, A.; DAO, T. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* 2023; versão revisada em 2024. Disponível em: https://arxiv.org/abs/2312.00752.

[3] BOURDOIS, L. *Introduction to State Space Models (SSM).* Hugging Face, 19 jul. 2024. Disponível em: https://huggingface.co/blog/lbourdois/get-on-the-ssm-train. Acesso em: 14 set. 2026.

[4] GU, A.; GOEL, K.; SAAB, K.; RÉ, C. *Structured State Spaces: Combining Continuous-Time, Recurrent, and Convolutional Models.* Hazy Research, 14 jan. 2022. Disponível em: https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3. Acesso em: 14 set. 2026.

[5] GU, A.; DAO, T.; ERMON, S.; RUDRA, A.; RÉ, C. *HiPPO: Recurrent Memory with Optimal Polynomial Projections.* NeurIPS, 2020. Disponível em: https://arxiv.org/abs/2008.07669.

[6] DAO, T.; FU, D. Y.; ERMON, S.; RUDRA, A.; RÉ, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS, 2022. Disponível em: https://arxiv.org/abs/2205.14135.